

Complete Technical Overview — *Taskmint*

What Is This Application?

Taskmint is a **cloud-native Business Management SaaS platform** built specifically for **Indian MSMEs (Micro, Small & Medium Enterprises)**. It allows a business owner to:

- Manage products and inventory
 - Run a **public-facing online storefront** (like a mini-Shopify)
 - Receive and manage customer orders
 - Generate **GST-compliant invoices**
 - Build **custom databases and forms** (like a mini-Airtable)
 - Automate workflows and announcements
 - View business dashboards/analytics
 - Send order notifications via **Gmail**
-

Architecture Overview

This is a **Full-Stack Web Application** using a **Client-Server architecture**:

[Customer Browser / MSME Browser]



[React Frontend] ← Vite Dev Server / Production Build

↕ HTTP REST API

[Flask Backend] ← Python



[SQLite Database] ← File-based DB (msme_builder.db)

Languages Used

Language	Where	Why
JavaScript (JSX)	Frontend (/frontend/src)	Powers all UI logic and components
Python 3	Backend (/backend)	Server-side logic, API, DB operations
CSS	index.css	Global styling, animations, themes
HTML	Embedded inside JSX	React compiles JSX to HTML — it IS HTML but written in a superset syntax
SQL	Internally via SQLAlchemy	Database queries generated by the ORM

? Why React and Not Plain HTML/CSS?

You might think: *"Why not just write an HTML file with some JavaScript?"* Here's why React was chosen:

The Problem with Plain HTML

A plain HTML page is **static** — once the browser downloads it, the page has to **fully reload** every time something changes (like updating the cart, or switching pages). This creates:

- Slow UX (full page reloads)
- Code that becomes impossible to maintain as it grows
- No reusable components

What React Solves

Problem	React Solution
Full page reloads	SPA (Single Page Application) — only the data changes, not the whole page
Code duplication	Reusable Components (e.g., OrderCard is one component used everywhere)
Managing UI state	useState / useEffect hooks — e.g., show spinner while loading orders

Problem	React Solution
Page navigation	react-router-dom gives client-side navigation without page reload
Complex interactions	React handles conditional rendering, modals, form state seamlessly

Bottom line: HTML + CSS is the final output, but React is the *tool* that builds and manages that output **intelligently and efficiently**.

Frontend — All Modules Used

Located in d:/TNImpact/frontend

Framework & Build Tool

Module	Purpose
React 19	Core UI library — builds the interface as components
Vite 7	Lightning-fast development server and build tool (replaces Webpack)
react-dom	Renders React components into the actual browser DOM

Routing

Module	Purpose
react-router-dom v7	Client-side routing (/ → HomePage, /products → ProductsPage, etc.)

Styling

Module	Purpose
TailwindCSS v4	Utility-first CSS classes for rapid, consistent styling
@tailwindcss/vite	Vite plugin integration for Tailwind v4
Google Fonts (Outfit)	Modern typeface loaded via CSS @import

UI Icons

Module	Purpose
lucide-react	Beautiful SVG icon library (Home, ShoppingCart, Trash2, etc.)

Authentication

Module	Purpose
@react-oauth/google	Google OAuth 2.0 login button integration
jwt-decode	Decodes the Google JWT token on the frontend to extract user email/name

API Communication

Module	Purpose
Native fetch() API	Makes HTTP calls to the Flask backend REST API (no extra library needed)
axios	Also available as an alternative HTTP client (installed but primarily fetch is used)

Charts & Visualization

Module	Purpose
chart.js	Core charting engine
react-chartjs-2	React wrapper for Chart.js (used in DashboardPage)

Drag & Drop

Module	Purpose
react-dnd	Drag-and-drop system for the VisualBuilderPage (rearranging form fields)
react-dnd-html5-backend	HTML5 drag-and-drop backend for react-dnd

PDF & Invoice

Module	Purpose
jsPDF	Generates PDF invoices programmatically from InvoicePrint.jsx

⚙️ Backend — All Modules Used

Located in d:/TNImpact/backend

Framework

Module	Purpose
Flask	Lightweight Python web framework — handles HTTP routes and responses
flask-cors	Adds CORS headers so the React frontend (port 5173) can call the backend (port 5000) without browser security errors

Database ORM

Module	Purpose
Flask-SQLAlchemy	Object-Relational Mapper (ORM) — lets you write Python classes instead of raw SQL
SQLite (via SQLAlchemy)	The actual database engine

Email

Module	Purpose
smtplib (Python standard library)	Sends order notification emails via Gmail SMTP
email.mime	Formats HTML/text email bodies

Utilities

Module	Purpose
os	File system paths (for locating the SQLite DB file)
datetime	Timestamps for orders, created_at fields
secrets / uuid	Generates unique order numbers

Database: Why SQLite?

The application uses **SQLite** as its database, stored in a single file: `msme_builder.db`.

What Is SQLite?

SQLite is a **serverless, file-based relational database**. It doesn't need a separate database server process running — the entire database lives in one `.db` file.

Why SQLite for This Project?

Reason	Explanation
Zero configuration	No server to install, no ports to configure. Just a file.
Ideal for MSMEs	Single-tenant apps with moderate data volume — SQLite handles this perfectly
Easy portability	The entire database is one file — easy to back up, move, or export
Flask-SQLAlchemy compatible	Seamlessly swappable with PostgreSQL or MySQL if scaling is needed
No operational overhead	A small business doesn't need a DBA managing database servers

When Would It Become a Problem?

SQLite struggles with **concurrent writes** (multiple users writing at the same time). If this scales to thousands of users, the `DATABASE_URI` in `app.py` would simply be changed to PostgreSQL — the rest of the code stays the same.

Database Tables (Models)

Table	Purpose
<code>products</code>	Product catalog (name, price, stock, GST slab, GST type)
<code>orders</code>	Order header (customer info, total, status, timestamps)
<code>order_items</code>	Line items inside each order (product, qty, price, GST breakdown)
<code>order_tracking</code>	Status history for each order (placed → accepted → shipped → completed)
<code>users</code>	Business owner accounts (email, hashed password, company name)

Table	Purpose
forms	Custom database table definitions (built in Database Builder)
form_fields	Column definitions for each custom form
form_records	Actual rows of data in custom forms
record_values	Individual cell values in custom form records
workflows	Automation rules (trigger → action) defined by users

Pages and What They Do

File	Page	Purpose
LoginPage.jsx	/login	Authentication (email/password + Google OAuth)
HomePage.jsx	/	Dashboard showing products, store link, and custom tables
ProductsPage.jsx	/products	Add/edit/delete products with GST settings
ActiveOrdersPage.jsx	/orders	View, filter, search, accept/ship/complete orders; view invoices
CustomerStorePage.jsx	/store	Public-facing storefront for customers to browse and place orders
DashboardPage.jsx	/dashboard	Revenue charts and business analytics
VisualBuilderPage.jsx	/builder	Drag-and-drop custom database form builder
DataPage.jsx	/data/:formId	View and add records to custom database tables
WorkflowPage.jsx	/workflows	Create trigger-based automations
AnnouncementsPage.jsx	/announcements	Send WhatsApp/email announcements to customers

File	Page	Purpose
SettingsPage.jsx	Settings	Email configuration, logo URL, account settings

vs Comparison: Taskmint vs. ZohoBooks vs. Tally

Feature	Taskmint	ZohoBooks	Tally Prime
Type	Web SaaS (browser-based)	Web SaaS	Desktop software
Platform	Any browser, any OS	Any browser, any OS	Windows only
Primary Users	Small/micro MSMEs	SMEs to Mid-market	Indian businesses of all sizes
GST Compliance	✓ CGST/SGST/IGST, custom slabs	✓ Full GST filing, returns	✓ Industry gold standard
GST Filing (Returns)	✗ Not supported	✓ GSTR-1, 3B auto-generation	✓ GSTR-1, 3B, reconciliation
Invoice Generation	✓ PDF/Print Tax Invoice	✓ Professional branded invoices	✓ Full accounting invoices
Inventory Management	✓ Basic (stock tracking)	✓ Advanced (batches, warehouses)	✓ Very advanced
Customer Storefront	✓ Built-in public store link	✗	✗
Custom Database Builder	✓ (no-code drag-and-drop)	✗	✗
Workflow Automation	✓ Basic automations	✓ Advanced workflow rules	✗

Feature	Taskmint	ZohoBooks	Tally Prime
Analytics Dashboard	✓ Basic charts	✓ Advanced reports	✓ Very advanced reports
Order Management	✓ Full (pending→delivered)	✓ Sales orders	✓ Sales orders
Accounting (Ledgers, P&L)	✗	✓ Full double-entry accounting	✓ Full accounting
Multi-user	✗ (single business owner)	✓ Role-based access	✓ Multi-user
Mobile App	✗ (mobile-responsive web)	✓ iOS + Android app	✓ Tally on Mobile
Pricing	Free / Self-hosted	₹749–₹1,499/month	₹18,000–₹54,000 one-time
Internet Required	✓ Always	✓ Always	✗ Works offline
Email Notifications	✓ Gmail-powered	✓ Built-in	✗
Open Source / Customizable	✓ Fully customizable	✗ Closed source	✗ Closed source
Tech Stack	React + Flask + SQLite	Proprietary (Java-based)	Proprietary (C/C++)

? Expected Questions & Answers

🔧 Technical Questions

Q1: Why did you use REST API instead of GraphQL?

REST is simpler to implement with Flask, and for an MSME app with well-defined resources (products, orders, users), REST endpoints are predictable and easy to debug. GraphQL adds complexity that isn't justified for this scale.

Q2: Why SQLite and not MySQL or PostgreSQL?

For a single-tenant MSME application, SQLite is sufficient. There are no concurrent heavy write operations and the data volumes are manageable. SQLAlchemy makes it trivial to migrate to PostgreSQL just by changing one line (DATABASE_URI) if needed.

Q3: How does authentication work?

Two methods: (1) **Email/Password** — credentials are checked against the users table; (2) **Google OAuth** — the frontend receives a JWT from Google, decodes it with jwt-decode to get the email, and sends it to /api/auth/google. The backend finds or creates the user.

Q4: How is GST calculated?

Products have a gst_slab (e.g., 18 for 18%) and a gst_type (cgst_sgst for intra-state, igst for inter-state). The backend computes: $\text{GST Amount} = (\text{Price} \times \text{Qty} \times \text{Rate}) / 100$. CGST and SGST are each half of the total (for intra-state).

Q5: How does the custom database builder work?

The builder lets users define tables (Form model) with columns (FormField model). Records are stored as FormRecord → RecordValue pairs. This is an Entity-Attribute-Value (EAV) pattern — flexible enough to store any schema without code changes.

Business/Product Questions

Q6: How is this different from a regular invoice tool?

Taskmint combines a public storefront, order management, inventory, invoice generation, custom database builder, and workflow automation in one tool. A regular invoice tool only generates invoices — it doesn't manage orders or products.

Q7: Who is the target user?

A micro-business owner (kiranas, artisans, boutiques, local manufacturers) who sells online, needs GST-compliant bills, wants to track orders and inventory, but cannot afford ₹1,500/month for Zoho or ₹50,000 for Tally.

Q8: What is the Unique Selling Point (USP)?

Three USPs: (1) Built-in public storefront — customers can order directly. (2) No-code custom database builder — build any internal record system without a developer. (3) Free, open-source, and self-hostable.

Q9: Is it scalable?

The frontend can be deployed on any CDN (Netlify, Vercel). The backend can be containerized (Docker) and deployed on any cloud. The database can be migrated from SQLite to PostgreSQL by changing one config line. Yes — it's designed to scale.

Q10: Why not use an existing framework like Odoo?

Odoo is extremely complex, expensive to customize, and requires significant IT expertise to operate. Taskmint is intentionally lightweight, focused, and usable by someone with zero technical background.

Q11: Why is React used? Is HTML not sufficient?

HTML + CSS alone is sufficient for *static* pages (like a brochure website). But Taskmint has dozens of **interactive states** — a cart that updates without reload, modals that appear/disappear, live order status updates, charts that redraw with new data. All of this requires JavaScript to dynamically modify the page. React provides a structured, efficient way to manage this complexity through components, state management, and a virtual DOM that minimizes unnecessary page updates. So HTML IS used — React just generates and manages it smarter.